

# Sui RPC Migration Plan

---

Full migration off JSON-RPC, onto gRPC (primary) + GraphQL (paginated reads)

Talise Engineering

2026-05-28

## Executive summary

---

Talise's Sui RPC surface is **35 gRPC sites + 17 JSON-RPC sites + 1 iOS direct JSON-RPC fallback + 1 web-frontend JSON-RPC site = 54 call sites total**. All five **hot paths** (per-send / per-request) already use gRPC; the JSON-RPC sites are intentional fallbacks for endpoints whose response shape isn't available on gRPC.

This document is the plan to:

1. Migrate every backend JSON-RPC call to **gRPC** (point reads, executions, lookups) or **GraphQL** (paginated history, multi-entity reads).
2. Build a native **iOS gRPC SDK** so the app no longer needs a JSON-RPC fallback.
3. Move the single web-frontend JSON-RPC site to GraphQL.
4. Delete the JSON-RPC client entirely and add a CI gate against its return.
5. Cover every migrated site with an integration test.

**Total estimated effort:** ~7.5 engineer-days, broken into 5 phases, broken into 50 sub-plans across the phases.

# Architecture decision

---

| Transport       | Use for                                                                   | Why                                                                                |
|-----------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>gRPC</b>     | Point reads, tx execution, simulation, Suins lookup, <b>all hot paths</b> | Lowest latency, typed contracts, native streaming via <code>subscribeEvents</code> |
| <b>GraphQL</b>  | Activity feed, multi-entity profile reads, paginated history              | Cursor pagination, multi-entity joins in one round-trip                            |
| <b>JSON-RPC</b> | ✗ Removed                                                                 | Legacy; both gRPC and GraphQL cover the surface we need                            |

The current `lib/sui.ts` exports a `sui()` returning `SuiGrpcClient` and a `suiJsonRpc()` returning `SuiJsonRpcClient`. After migration, only `sui()` and `suiGraphQL()` survive.

# Endpoint inventory

---

## Mainnet

| Transport     | URL                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------|
| gRPC          | <code>https://fullnode.mainnet.sui.io:443</code>                                                         |
| GraphQL       | <code>https://graphql.mainnet.sui.io/graphql</code>                                                      |
| Archival gRPC | <code>archive.mainnet.sui.io:443</code> (only when historical retention exceeds fullnode pruning window) |

## Testnet

| Transport | URL                                                 |
|-----------|-----------------------------------------------------|
| gRPC      | <code>https://fullnode.testnet.sui.io:443</code>    |
| GraphQL   | <code>https://graphql.testnet.sui.io/graphql</code> |

Production warning: public endpoints are rate-limited. Before Phase 4 deploys, we need either a self-hosted Sui full node or a provider endpoint.

## Phase 0 — Pre-work (1/2 day)

---

Before any migration code lands.

1. **Protobuf vendoring.** Pull `crates/sui-rpc-api/proto/*.proto` from `MystenLabs/sui` at the version matching our `@mysten/sui` package (currently `^2.16`). Vendor into `ios/Talise/Network/SuiProto/proto/` and `web/lib/sui-proto/` for traceability.
2. **GraphQL singleton.** Add `lib/sui-graphql.ts` exporting a `suiGraphQL()` helper that returns a cached `SuiGraphQLClient` for the current network.
3. **Feature flag scaffold.** Add `TALISE_LEGACY_JSONRPC` env var (default `1` during migration, flip to `0` per-site as each is verified). Lets us mid-migrate safely.
4. **CI gate.** Add a lint rule that fails CI if any new `import.*SuiJsonRpcClient` appears outside of files in the explicit allow-list (which shrinks to zero by Phase 5).
5. **Integration-test scaffolding.** Spin up a `web/__tests__/sui/` folder with a shared harness for hitting mainnet endpoints with read-only operations.

## Phase 1 — Backend: 17 JSON-RPC sites → gRPC / GraphQL (2 days)

| #  | Site                                    | Current method                       | Migrate to                                     | Notes                                       |
|----|-----------------------------------------|--------------------------------------|------------------------------------------------|---------------------------------------------|
| 1  | /api/sui/epoch                          | getLatestSuiSystemState              | gRPC (epoch via LedgerService)                 | Shared helper #2                            |
| 2  | /api/auth/mobile/start                  | getLatestSuiSystemState              | Same shared helper                             |                                             |
| 3  | /api/pk/status                          | getBalance                           | gRPC core.listBalances                         | Response structure changes — update reader  |
| 4  | /api/pk/status                          | getObject                            | gRPC core.getObject                            | Same                                        |
| 5  | /api/tx/record                          | getTransactionBlock                  | gRPC core.getTransaction                       | <b>Largest scope</b> verifier code rewrite  |
| 6  | /api/vault/record                       | getTransactionBlock                  | Same verifier helper as #5                     |                                             |
| 7  | /api/vault/migrate-confirm              | getTransactionBlock                  | Same                                           |                                             |
| 8  | /api/vault/repoint-confirm              | getTransactionBlock                  | Same                                           |                                             |
| 9  | /api/cron/auto-swap-sweep event polling | queryEvents paginated                | <b>GraphQL</b> — Relay cursor                  | gRPC has subscription only, no cursor query |
| 10 | /api/vault/state Bag reads              | getObject({showContent})             | gRPC core.getObject({include: {content:true}}) | Bag is under content.json                   |
| 11 | lib/activity.ts                         | queryEvents + queryTransactionBlocks | <b>GraphQL</b>                                 | Single query replaces 2-3 round-trips       |
| 12 | lib/suins-lookup.ts (handle)            | getOwnedObjects paginated            | gRPC core.listOwnedObjects                     | Cursor pagination ops                       |
| 13 | lib/suins-lookup.ts (subname)           | getOwnedObjects                      | gRPC core.listOwnedObjects                     |                                             |
| 14 | lib/deepbook-margin.ts                  | getOwnedObjects                      | gRPC core.listOwnedObjects                     |                                             |

| #  | Site                  | Current method          | Migrate to                      | Notes |
|----|-----------------------|-------------------------|---------------------------------|-------|
| 15 | lib/zkclient.ts       | executeTransactionBlock | gRPC<br>core.executeTransaction |       |
| 16 | lib/zkclient.ts       | getCoins                | gRPC core.listCoins             |       |
| 17 | lib/suins-operator.ts | executeTransactionBlock | gRPC<br>core.executeTransaction |       |

**Top risk.** `getTransaction` response shape on gRPC differs from JSON-RPC. The verifier in `/api/tx/record` reads `transaction.data.sender` + `effects.status` (JSON-RPC paths). Mitigation: write a `normalizeTransactionShape()` helper that maps either source into a shared TS type, so the verifier doesn't care which transport was used.

**Test gate.** Each migrated site gets an integration test against mainnet read data + an existing-flow regression run.

## Phase 2 — Web frontend (1/2 day)

---

The web frontend has exactly one direct Sui call: `web/components/FixSubnameBanner.tsx` (Suins subname repair flow). Migrate it to GraphQL via `@mysten/sui/graphql`:

```
const client = new SuiGraphQLClient({
  network: "mainnet",
  url: "https://graphql.mainnet.sui.io/graphql",
});
const data = await client.query({
  query: graphql(`query { ... }`),
  variables: { ... },
});
```

Everything else in the web app is server-side and covered by Phase 1.

## Phase 3 — iOS gRPC SDK (3 days) — NEW

---

The biggest single piece of work. iOS currently has one direct Sui call

( `ZkLoginCoordinator.fetchEpochViaMainnetRPC` , raw JSON-RPC POST). To remove JSON-RPC from iOS, we build a thin Swift gRPC client over Apple's `grpc-swift` v2 + Sui's protobuf definitions.

### Steps:

1. **Swift Package dependencies.** Add to `ios/project.yml` :

```
packages:
  GRPC:
    url: https://github.com/grpc/grpc-swift
    majorVersion: 2.0.0
  SwiftProtobuf:
    url: https://github.com/apple/swift-protobuf
    majorVersion: 1.0.0
```

2. **Generate Swift bindings.** From the .proto files vendored in Phase 0:

```
protoc \
  --plugin=protoc-gen-grpc-swift \
  --plugin=protoc-gen-swift \
  --grpc-swift_out=ios/Talise/Network/SuiProto/Generated \
  --swift_out=ios/Talise/Network/SuiProto/Generated \
  ios/Talise/Network/SuiProto/proto/*.proto
```

Generated files are committed to the repo so iOS builds don't require `protoc` locally.

3. **Build `SuiGrpcClient.swift`** . A Swift class that mirrors the subset of methods iOS needs:

```
@MainActor
final class SuiGrpcClient {
  static let shared = SuiGrpcClient(
    baseUrl: "https://fullnode.mainnet.sui.io:443"
  )
  func getLatestEpoch() async throws -> UInt64
  func getReferenceGasPrice() async throws -> UInt64
  func getBalance(owner: String, coinType: String) async throws -> Balance
  func executeTransaction(bytes: Data, signatures: [String]) async throws -> TxResult
}
```

We only expose the 4 methods iOS calls today + the 1 we want to switch to ( `getLatestEpoch` ). The class can grow as iOS picks up more direct Sui usage (offline tx assembly, wallet adapter, etc.).

4. **Migrate the single existing iOS direct call.** `ZkLoginCoordinator.fetchEpochViaMainnetRPC` (line 471-489) is replaced by `SuiGrpcClient.shared.getLatestEpoch()` . The



```
URLSession.shared.data(for: req) raw POST disappears.
```

5. **Unit tests.** XCTest cases for each method that round-trip a known-good mainnet object.

**Risks.**

- grpc-swift v2 is recent; learning curve on Swift concurrency + grpc-web transport
- Protobuf code generation needs consistent plugin versions across our laptops and CI; commit generated code to side-step this
- The Sui gRPC server uses Connect gRPC over HTTP/2 — make sure the iOS networking stack handles ALPN correctly

**Mitigation.** Scope the iOS SDK to 4 methods total. Generated code is checked in. Failing fast on any protobuf field-naming mismatch via unit tests.

## Phase 4 — Test matrix (1 day)

---

| Test                                | Surface                                                                                     | Pass criteria                                 |
|-------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------|
| <code>send-gasless.test.ts</code>   | <code>/api/send/sponsor-prepare</code> + <code>/api/send/gasless-submit</code>              | Digest returned, on-chain receipt visible     |
| <code>send-sponsored.test.ts</code> | <code>/api/send/sponsor-prepare</code> (round-up on) + <code>/api/zk/sponsor-execute</code> | Send + supply leg confirmed                   |
| <code>supply.test.ts</code>         | <code>/api/earn/supply/prepare</code> + <code>sponsor-execute</code>                        | NAVI position visible                         |
| <code>withdraw.test.ts</code>       | <code>/api/earn/withdraw/prepare</code> + <code>sponsor-execute</code>                      | Funds back in wallet                          |
| <code>vault-state.test.ts</code>    | <code>/api/vault/state</code>                                                               | Returns balances + supplied amount            |
| <code>activity.test.ts</code>       | <code>lib/activity.ts</code> via GraphQL                                                    | Feed renders mixed entries correctly          |
| <code>tx-record.test.ts</code>      | <code>/api/tx/record</code> verifier                                                        | Verifies a known mainnet digest               |
| <code>ios-epoch.test</code>         | Swift unit test on <code>SuiGrpcClient.getLatestEpoch</code>                                | Returns epoch $\geq$ current                  |
| <code>prod-smoke</code>             | \$0.01 USDsui send on prod                                                                  | Send → Withdraw → Supply → Activity all green |

## Phase 5 — JSON-RPC removal (1½ day)

---

Final cleanup, gated on Phases 1-4 green.

- Delete `suiJsonRpc()` from `lib/sui.ts`
- Drop `SuiJsonRpcClient` imports from `lib/zkclient.ts`, `lib/navi-supply.ts`, `components/FixSubnameBanner.tsx`
- Drop `@mysten/sui/jsonRpc` import surface
- Remove iOS's `fetchEpochViaMainnetRPC` (replaced by `SuiGrpcClient.shared.getLatestEpoch()` in Phase 3)
- Final PR diff sweep: zero matches for `JsonRpc` outside of the deprecation comment in CI lint
- Update `CLAUDE.md` and `docs/architecture/` to reflect gRPC-primary stance

## Cost + ordering

---

| Phase                      | Days             | Blocks      | Parallelism with                       |
|----------------------------|------------------|-------------|----------------------------------------|
| 0. Pre-work                | 0.5              | —           | Independent — kickoff                  |
| 1. Backend gRPC + GraphQL  | 2                | 0           | Internal parallel by call-site cluster |
| 2. Web<br>FixSubnameBanner | 0.5              | 0           | Phase 1 + Phase 3 (fully independent)  |
| 3. iOS gRPC SDK            | 3                | 0           | Phase 1 + Phase 2 (independent)        |
| 4. Test matrix             | 1                | Phase 1 + 3 | Tests can be drafted alongside         |
| 5. JSON-RPC removal        | 0.5              | All         | Sequential, last                       |
| <b>Total</b>               | <b>~7.5 days</b> |             |                                        |

# Pre-conditions for execution

---

Before starting Phase 1:

1. **Approve scope.** Confirm 7.5 days is acceptable, or tell me which phases to defer.
2. **Self-hosted full node decision.** Public endpoints rate-limit; production needs either Mysten's paid endpoint or a self-hosted fullnode. Decide before Phase 4 deploys.
3. **executeTransaction transport.** Currently gRPC. GraphQL also has it via mutation. Recommend staying on gRPC for hot path (lower latency, no read-after-write nested scope needed).
4. **Ordering.** Phase 1 (backend) and Phase 3 (iOS SDK) are independent — they can start in parallel. Phase 1 gives the bigger immediate code-quality win; Phase 3 unlocks future offline iOS features.

## Appendix A — JSON-RPC → gRPC method map

---

| JSON-RPC                                   | gRPC equivalent                                               | Available?                              |
|--------------------------------------------|---------------------------------------------------------------|-----------------------------------------|
| <code>sui_getBalance</code>                | <code>core.listBalances</code>                                | ✓                                       |
| <code>sui_getCoins</code>                  | <code>core.listCoins</code> (with cursor)                     | ✓                                       |
| <code>sui_getObject</code>                 | <code>core.getObject</code> (with <code>include</code> flags) | ✓                                       |
| <code>sui_multiGetObjects</code>           | <code>core.getObjects</code>                                  | ✓                                       |
| <code>sui_getTransactionBlock</code>       | <code>core.getTransaction</code>                              | ✓ (different shape)                     |
| <code>sui_executeTransactionBlock</code>   | <code>core.executeTransaction</code>                          | ✓                                       |
| <code>sui_dryRunTransactionBlock</code>    | <code>core.simulateTransaction</code>                         | ✓                                       |
| <code>sui_queryEvents</code>               | <code>subscriptionService.subscribeEvents</code> (streaming)  | Streaming only;<br>historical → GraphQL |
| <code>sui_queryTransactionBlocks</code>    | n/a (no cursor on gRPC)                                       | GraphQL only                            |
| <code>sui_getOwnedObjects</code>           | <code>core.listOwnedObjects</code> (with cursor + filter)     | ✓                                       |
| <code>sui_getLatestSuiSystemState</code>   | gRPC <code>LedgerService</code> epoch helper                  | ✓ (different shape)                     |
| <code>sui_resolveNameServiceAddress</code> | <code>nameService.forwardLookupName</code>                    | ✓                                       |
| <code>sui_resolveNameServiceNames</code>   | <code>nameService.reverseLookupName</code>                    | ✓                                       |

## Appendix B — Endpoint contention + rate limits

---

Public Sui endpoints ( `fullnode.*.sui.io` , `graphql.*.sui.io` ) return `RESOURCE_EXHAUSTED` (gRPC) or `429` (GraphQL) under load. For production:

- Use Mysten's paid endpoints (Sui Foundation), OR
- Self-host a full node + GraphQL service (operationally heavier but no rate limit)

Decision is needed before Phase 4 ships to production. For Phase 1-3 development against mainnet, public endpoints are fine.

## Appendix C — Parallelism caveat

---

This plan is broken into 50 sub-plans (separate document). **Realistic max concurrent agents is ~6–8**, not 50. Reasons:

- File contention: many sub-plans modify `lib/sui.ts`, `lib/activity.ts`, or `ZkLoginCoordinator.swift`. Agents stepping on each other will produce merge conflicts.
- Sequential dependencies: Phase 3's Swift bindings must finish before any iOS sub-plan can use them; the `getTransaction` verifier rewrite must finish before the four `/api/vault/*` migration sub-plans land.
- Shape alignment: all sub-plans touching the same response-shape change must agree on the same normalized type.

The 50-sub-plan document tags each sub-plan as **PARALLEL-SAFE**, **SEQUENTIAL-WITHIN-PHASE**, or **BLOCKED-BY-OTHER**. Spawn agents accordingly.